

Sprint 4 — Builder Pattern

Class 2

Lesson 6 — Immutable Objects

Learning Objective

By the end of this lesson, we should understand:

- What is Immutable Object?
- Why immutable objects are safer.
- Why Builder is commonly used with immutable objects.

Start with a Question

Can someone accidentally change this Room?

```
Room room = new Room();
```

```
room.setPrice(BigDecimal.valueOf(50));
```

```
...
```

```
room.setPrice(BigDecimal.ZERO);
```

Answer: Yes.

The object changes anytime.

Real Enterprise Example

Imagine a payment.

Payment

id

amount

currency

payer

receiver

status

createdDate

Should someone be allowed to do this?

```
payment.setAmount(BigDecimal.ZERO);
```

Absolutely not.

Once a payment is created, it should never change.

Immutable Object

Instead of setters:

```
public class Payment {  
  
    private final UUID id;  
    private final BigDecimal amount;  
    private final Currency currency;  
  
}
```

Notice:

final

No setters.

No modification.

Builder + Immutable Object

```
public class Payment {  
  
    private final UUID id;  
    private final BigDecimal amount;  
  
    private Payment(Builder builder) {  
        this.id = builder.id;  
        this.amount = builder.amount;  
    }  
  
    public static class Builder {  
        private UUID id;  
        private BigDecimal amount;  
  
        public Builder id(UUID id) {  
            this.id = id;  
            return this;  
        }  
  
        public Builder amount(BigDecimal amount) {  
            this.amount = amount;  
            return this;  
        }  
  
        public Payment build() {  
            return new Payment(this);  
        }  
    }  
}
```

Usage

```
Payment payment = new Payment.Builder()
```

```
.id(UUID.randomUUID())  
.amount(BigDecimal.valueOf(100))  
.build();
```

Now

```
payment.setAmount(...)
```

Impossible.

Why Enterprise Loves Immutable Objects

Benefits:

- Thread-safe
- Easier debugging
- No accidental modification
- Predictable behavior

Lesson 7 — Builder Validation

Why It Exists

Builder should not create invalid objects.

Suppose Room requires:

roomCode

price

Bad Builder

```
Room room = new Room.Builder().build();
```

Oops.

Invalid Room.

Validate Inside build()

```
public Room build() {  
  
    if (roomCode == null || roomCode.isBlank()) {  
        throw new IllegalArgumentException("Room code is required");  
    }  
  
    if (price == null) {  
        throw new IllegalArgumentException("Price is required");  
    }  
  
    return new Room(this);  
}
```

Now

```
new Room.Builder().build();
```

Immediately fails.

Why Validation Belongs Here

Where is the last place before object creation?

Answer: build()

It is a perfect place.

Lesson 8 — Reusable Builder

Many developers don't know Builder can be reused.

Example

```
Room.Builder builder = new Room.Builder()  
    .price(BigDecimal.valueOf(50))  
    .wifi(true);
```

Build Deluxe

```
Room deluxe = builder  
    .roomCode("DLX001")  
    .roomName("Deluxe")  
    .build();
```

Build Suite

```
Room suite = builder  
    .roomCode("STE001")  
    .roomName("Suite")  
    .build();
```

Shared configuration.

Lesson 9 — Lombok Builder

Now we finally understand

WHY Lombok exists.

Instead of writing

200 lines

We write

@Builder

@Getter

```
public class Room {  
  
    private UUID id;  
  
    private String roomCode;  
  
    private String roomName;  
  
    private BigDecimal price;  
  
}
```

Usage

```
Room room = Room.builder()  
    .roomCode("DLX001")  
    .roomName("Deluxe")  
    .price(BigDecimal.valueOf(50))  
    .build();
```

Some students often think @Builder is magic.

Actually, Lombok generates exactly the Builder class we learned.

Nothing magical.

Lesson 10 — Builder in Java

Builder is everywhere.

Example 1

StringBuilder

```
String message = new StringBuilder()  
    .append("Hello ")  
    .append("Builder ")  
    .append("Pattern")  
    .toString();
```

Example 2

Java HttpClient

```
HttpRequest request = HttpRequest.newBuilder()  
    .uri(URI.create("https://google.com"))  
    .GET()  
    .build();
```

Example 3

ProcessBuilder

```
ProcessBuilder builder =  
    new ProcessBuilder(  
        "java",  
        "-version");
```

Java itself loves Builder.

Lesson 11 — Builder in Spring

Spring uses Builder extensively.

WebClient

```
WebClient client = WebClient.builder()
    .baseUrl("http://localhost:8080")
    .defaultHeader("Authorization", token)
    .build();
```

UriComponentsBuilder

```
String url =
    UriComponentsBuilder
        .fromPath("/rooms")
        .queryParams("page", 1)
        .queryParams("size", 20)
        .build()
        .toUriString();
```

ResponseEntity

```
return ResponseEntity
    .ok()
    .body(roomResponse);
```

Builder-like API.

We are already using Builder.

Now they know its name.

Lesson 12 — Common Mistakes

Mistake 1

Builder for everything

Bad

```
UserId.builder()
```

Seriously?

Just use

```
new UserId(...)
```

Builder is for complex objects.

Mistake 2

Mutable Builder Product

```
@Builder
```

```
@Setter
```

Then

```
room.setPrice(...)
```

We lose immutability.

Mistake 3

Skipping Validation

Never allow

```
Room.builder()  
    .build();
```

if required fields are missing.

Mistake 4

Builder with only two fields

```
Money.builder()  
    .amount(...)  
    .currency(...)
```

Probably unnecessary.

Enterprise Example

Room Platform

Instead of

```
Room room =  
    new Room(  
        id,  
        propertyId,  
        roomCode,  
        roomName,  
        roomType,  
        maxAdult,  
        maxChild,  
        area,  
        floor,  
        price,  
        currency,  
        wifi,  
        kitchen,  
        parking,
```

```
        balcony,  
        description,  
        active,  
        roomStatus  
    );
```

Use

```
Room room =  
    Room.builder()  
        .propertyId(propertyId)  
        .roomCode("DLX001")  
        .roomName("Deluxe")  
        .roomType(RoomType.DELUXE)  
        .maxAdult(2)  
        .maxChild(1)  
        .price(BigDecimal.valueOf(50))  
        .currency("USD")  
        .wifi(true)  
        .kitchen(false)  
        .parking(true)  
        .description("Sea view")  
        .active(true)  
        .roomStatus(RoomStatus.DRAFT)  
        .build();
```

Homework Project

Hotel Reservation Builder

Design these immutable classes using Builder:

Guest

id

firstName

lastName
phone
email
nationality
passportNumber
vip

Room

roomCode
roomType
price
currency
maxAdult
maxChild
wifi
balcony

Reservation

reservationNo
guest
room
checkIn
checkOut
numberOfGuests
specialRequest
status

Tasks

1. Make all classes immutable (final fields, no setters).
2. Implement validation in every build() method.
3. Use Lombok @Builder for one class and write a manual Builder for another. Compare the generated code and discuss the trade-offs.
4. Create a Reservation object using the nested builders:

```

Reservation reservation =
    Reservation.builder()
        .reservationNo("RES-2026-0001")
        .guest(
            Guest.builder()
                .firstName("John")
                .lastName("Smith")
                .email("john@example.com")
                .vip(true)
                .build()
        )
        .room(
            Room.builder()
                .roomCode("DLX001")
                .price(BigDecimal.valueOf(80))
                .currency("USD")
                .build()
        )
        .build();

```

Builder Pattern Summary

Problem	Solution
Too many constructor parameters	Builder
Constructor overload explosion	Builder
Readability	Fluent API
Invalid object creation	Validation in <code>build()</code>
Mutable objects	Immutable + Builder
Complex enterprise DTOs/entities	Builder
Reduce boilerplate	Lombok <code>@Builder</code>